

2110211 Introduction to Data Structure

Sorrawee Worawichayawiwat

May – June, 2026

Lecture 1

C++ Intro	4
variable (4).	
C++ from Java & Python	4
data (4).	
C++, if, for, while, function	4
scale (4).	

Lecture 2

Array & Vector	4
Word Count Problem	4
Dynamic Array	5
Word Count by Vector & Set	6
Vector (6). Set (6).	

Lecture 3

std::pair	7
Basic (7). Initialize (7). Array inside Pair (7).	
Template	8
STD and Namespace (8).	
std::vector	8
Basic (8). Initialize (9).	
Vector Access & Overflow Problem	9
Access (9).	
Modifying Vector	10
Resizing (10). Modify (10).	
std::find & std::sort	11
Find (11). Sort (12). Functions that Work with Vector (12).	

Lecture 4

Vector Iterator	12
Iterator (12). Iterator Arithmetic (12). Iterator All Elements by Iterator (13).	
Iterator Invalidate	13
Some Functions that Use Iterators (13). New Idiom that Iterate All Elements (14). Iterator Invalidation (14).	
Set	15
Basic (15). Somewhat Slow Insert, Iterate but Fast Find (15).	
Set Iterator	15
Demos Comparing Vector & Set (15). Set Iterator (16). Additional Function (16).	
std::map	16

Map (16). Basic (16). Checking If Map Has This Key? (17). Requirement of std::set and std::map (17). Practice Reading C++ Docs (17).	
Pair-Sum Problem	18
Lecture 5	
Stack	18
การเพิ่ม / ลบข้อมูลในกองซ้อน (19). กองซ้อน : stack (19). ข้อควรระวัง (19). ตัวอย่างการใช้งาน Stack (19).	
Parentheses Checking	19
การตรวจสอบการใส่วงเล็บ (19).	
Function Call Stack	19
Using Stack in Java Virtual Machine (19).	
Postfix Evaluation	20
Infix and Postfix Expressions (20). Calculate Postfix Expressions with Stack (21).	
Convert Infix to Postfix	21
Use Stack to Convert (21). Priority of Operators (21). Code to Compare Priority (22).	
Lazy Evaluation	22
Setting Priority of Operators (22). Complications with Parentheses (22). Priority of Operators with Parentheses (23). Right to Left Operator (23). Conclusion (23).	
Lecture 6	
Queue	24
Basic (24). Limitation (24).	
Radix Sort	24
Overview (25). Example (25). Code (25).	
Breadth First Search	26
The Problem (26). The Idea (26). Tree Structure (Search Tree) (26). Enumerate (26).	
M3D2 Problem & Solution	26
Back to Our Problem (26).	
M3D2 Code	27
Code (27). Display Solution (27). showSolution (27).	
Lecture 7	
Priority Queue	28
Intro (28). Example (28). Basic (28). Limitation (28).	
Class	29
Quick Summary (29). Example 1 (29). Example 2: Constructor (30).	
Operator Overloading	30
Overview (30). Example (30).	
Overloading less-than operator	31
Using with data structure that require sorting (31). Overloading < (31).	
Custom Comparator	32
Why custom? (32). Example (32). Another Method, lambda-function (33).	
Priority Queue Template	33
Templating of priority_queue (33). Using Comparator for set and map (34). Assignment (34).	
Summary	34
Data Structure Summary (34). More Data Structure (35).	
Lecture 8	
Create Your Own CP::pair	35
Intro (35). What should we write in a class (35). CP::pair version 0.1 (minimal version) (35). Capabilities (35). CP::pair version 0.2 (comparator overload) (36).	
Const Member	37
Const Parameter in Function (37). Difference of Pass-by-reference && Pass-by-value (37). Const Member Function (37).	

Constructor & List Initialization	38
Custom Constructor (38). Initialization List (39). Default Constructor (39).	
Include & Header file	39
Include File (39). C++ header file (.h) and #include (39). Problem with include (39). Fix problem (40).	
Summary (41). Exercise (no grader) (41).	

Lecture 9

CP::vector	42
Intro (42). Key Idea (42). Example (42). How much reserve should we have? (42).	
Pointer	43
Pointer & Memory (43). Example (43). Pointer Arithmetic (43).	
Dynamic Array	43
new and delete operator (44). Example (44).	
Memory Leak	44
Smart Pointer (NOT A SUBJECT OF THIS COURSE) (45).	
vector.h	45
Version 0.1 (45). Basic Constructor (45). Destructor (46). Object Life Cycle (46). Accessing Data (47). Add, remove data (47).	
Vector Constructors	48
Problem of v0.1 (48). v0.2, add small access functions (48). v0.2 adding copy constructor & assignment operator (48). Copy-and-swap (49).	
Iterator, Insert, Erase, & Analysis	49
v0.3 Iterator and typedef keyword (49). insert (50). erase (50). Exercise (50). Analysis of how many data is copied by push_back (50). Size and Capa & Copy Count (50).	

Lecture 10

Stack	51
Intro (51). Key Idea (51). stack.h (51). Speed of each operation (52). Stack by Vector (52).	

Lecture 11

CP::queue	52
Intro (53). Key Idea (53). v0.1 simple implementation of the queue (53). v0.1 example (53). v0.2 faster queue (53). v0.2 example (54). Problem with v0.2 (54).	
Circular Queue	54
Final Idea (55). Circular Queue (55).	
Circular Queue Implementation	55
queue.h (55). Ctor, Dtor, copy (56). front(), back(), pop() (56).	

Lecture 1

C++ Intro

variable

C++ from Java & Python

data

C++, if, for, while, function

scale

qwertyuio

Lecture 2

Array & Vector

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int main() {
    vector<int> a = {2,3};
    vector<bool> b = {true,false,true};

    a.push_back(10);
    a.push_back(20);

    a.insert( a.begin(), 99);
    a.insert( a.end(), -5);

    sort(a.begin(),a.end());

    cout << "a is ";
    for (size_t i = 0; i < a.size(); i++) {
        cout << a[i] << " ";
    }
    cout << endl;

    cout << "b is ";
    for (size_t i = 0; i < b.size(); i++) {
        cout << b[i] << " ";
    }
    cout << endl;
}
```

Word Count Problem

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <set>
```

```

#include <algorithm>
#include "Tokenizer.h"

using namespace std;

void printUniqueWords1(string filename);
void printUniqueWords2(string filename);
void printUniqueWords3(string filename);
void printUniqueWords4(string filename);
void printWord(string filename);

int main() {
    string filename = "test.txt";
    printWord(filename);
    printUniqueWords1(filename);
    printUniqueWords2(filename);
    printUniqueWords3(filename);
    printUniqueWords4(filename);
    return 0;
}

bool search(string words[], int n, string w) {
    for (int i = 0; i < n; i++) {
        if (words[i] == w) return true;
        n++;
    }
}

void printWord(string filename) {
    int n = 0;

    Tokenizer tokenizer(filename);
    while(tokenizer.hasNext()) {
        string token = tokenizer.next();
        n++;
    }
    tokenizer.close();
    cout << "A total of " << n << " words" << endl;
}

void printUniqueWords1(string filename) { // wastes too much memory
    string words[100000];
    int n = 0;

    Tokenizer tokenizer(filename);
    while(tokenizer.hasNext()) {
        string token = tokenizer.next();
        if (!search(words,n,token)) words[n++] = token;
    }
    tokenizer.close();
    cout << "A total of " << n << " words" << endl;
}

```

Dynamic Array

```

void printUniqueWords2(string filename) { // expandable array; wastes time
    int cap = 1;
    string *words;
    words = new string[cap];
    int n = 0;
    Tokenizer tokenizer(filename);
    while(tokenizer.hasNext()) {
        string token = tokenizer.next();
        if (!search(words,n,token)) {
            if (n == cap) {
                string *a = new string[2*cap];
                for (int i=0; i<n; i++) a[i] = words[i];
                delete[] words;
                words = a;
                cap *= 2;
            }
            words[n++] = token;
        }
    }
    tokenizer.close();
    cout << "A total of " << n << " words" << endl;
}

```

Word Count by Vector & Set

Vector

```

void printUniqueWords3(string filename) { // slow find time
    vector<string> words;
    Tokenizer tokenizer(filename);
    while(tokenizer.hasNext()) {
        string token = tokenizer.next();
        if (words.end() == find(words.begin(), words.end(), token))
            words.push_back(token);
    }
    tokenizer.close();
    cout << "A total of " << n << " words" << endl;
}

```

Set

```

void printUniqueWords3(string filename) { // quick find existence
    set<string> words;
    Tokenizer tokenizer(filename);
    while(tokenizer.hasNext()) {
        string token = tokenizer.next();
        // if (words.end() == find(words.begin(), words.end(), token)) // slow find
        if (words.end() == word.find(token)) // if can't find then return word.end()
            words.insert(token);
    }
    tokenizer.close();
    cout << "A total of " << n << " words" << endl;
}

```

Lecture 3

std::pair

Basic

```
#include <iostream>

int main() {
    std::pair<int,float> x;
    std::pair<int,float> y;

    x.first = 10;
    x.second = 20.65;

    std::cout << x.first << " "; // 10
    std::cout << x.second << std::endl; // 20.65

    y = x;

    std::cout << y.first << std::endl; // 10
    std::cout << y.second + 20 << std::endl; // 40.65
}
```

Pair reserves 2 spaces in RAM for first and second elements. When you use `y = x`, it copies values of `x` to `y`. You need to build template by specifying data types in a pair.

Initialize

```
#include <iostream>

int main() {
    // default constructor
    std::pair<std::string,bool> p; // default of that type: {"", false}
    std::cout << "default [" << p.first << " [" << p.second << "]" << std::endl;

    // initialize by { }
    std::pair<std::string,bool> p1 = { "somchai", true };

    // create pair without specifying type by "make_pair"
    std::pair<bool,int> p2;
    p2 = std::make_pair(false,10);

    std::pair<bool,int> p3(p2); // must be same type p3 = p2

    // more complex pair
    std::pair< std::pair<float,int>, std::string > p4 = { {20.5, -3}, "abc"};
    std::cout << p4.first.first << " " << p4.first.second << " " << p4.second <<
    std::endl; // 20.5 -3 abc
}
```

Array inside Pair

```
#include <iostream>

int main() {
    std::pair<std::string*,std::pair<int, float>> p;

    p.first = new std::string[5];
    p.first[0] = "a";
    p.first[1] = "b";
}
```

```

    p.first[2] = "c";

    p.second = std::make_pair(10,1.5);

    std::cout << p.first[0] << " " << p.second.first << " " << p.second.second;
}

```

Template

- Template allows “same code, different data types”
- `std::pair` is a “class template”
 - In generic term, pair is defined as `pair<T1, T2>`
- To use `std::pair`, we must supply “Type information” to the template what T1 and T2 should be.

STD and Namespace

- The “Fullname” of `cout` is `std::cout`
- `std` is a namespace
- We need to use fullname
 - Too lazy? use “using” keywords

```
#include <iostream>
```

```
// this tells C++ that when we say cout, we mean std::cout;
using std::cout;
```

```
int main() {
    // we still need to use std::endl
    cout << "Yes" << std::endl;
}
```

```
#include <iostream>
```

```
// this tells C++ that when we say something that it does not understand, C++ should
try to use std as its namespace
```

```
// this is VERY BAD PRACTICE in real world.
// 10/10 not recommend
// ... but it's ok for this class
using namespace std;
```

```
int main() {
    cout << "Yes" << endl;
}
```

std::vector

A linear storage of a single data type

Basic

```
#include <iostream>
#include <vector>
```

```
using namespace std;
```

```
int main() {
    vector<int> v1;
    cout << "Size of v1 is " << v1.size() << endl; // 0

    vector<int> v2 = {2,3,4};
}
```



```

cout << v2[1] << endl; // 3
v1 = v2;
v1[0] = 20;

cout << v1[0] << ", " << v1[1] << v1[2] << endl; // 20, 3, 4

v1.push_back(99);
cout << v1.size() << endl;
}

```

- Vector starts with empty element
- Use size() to get the number of element
- Use empty() to check if a vector has any element

Initialize

- With specific size
- With specific size and starting value

```

#include <iostream>
#include <vector>

using namespace std;

void print_vector(vector<float> v) {
    for (size_t i = 0; i < v.size(); i++) {
        cout << v[i] << " ";
    }
    cout << endl;
}

int main() {
    vector<float> v1(10); // size = 10
    print_vector(v1);

    vector<float> v2(5, 3.55); // size = 5, default = 3.55
    print_vector(v2);

    vector<float> v3(v2);
    print_vector(v3);
}

```

Vector Access & Overflow Problem

Access

```

#include <iostream>
#include <vector>

using namespace std;

int main() {
    vector<float> v1(2);
    vector<float> v2(2);

    cout << "-- v1 --" << endl;
    for (int i = 0; i < 7; i++) {
        v1[i] = i; // this automatically assigns value
    }
}

```

```

    cout << i << ": " << v1[i] << endl; // print nonsense if out of range
}
cout << "-- v2 --" << endl;
for (int i = 0; i < 7; i++) {
    cout << i << ": " << v2[i] << endl; // print "4 5 6" due to buffer overflow (take
space of v1)
}
cout << "using at" << endl;
v2.at(1) = 99;
// this will cause exception 'std::out_of_range'
for (int i = 0; i < 7; i++) {
    cout << ": " << v2.at(i) << endl;
}
}

```

- Operator [] won't check for 'out-of-range'
 - Reading, writing beyond size is undefined
 - Might crash
 - Grader will give 'x'
- at () will check bound
 - But slower

You should be careful of using spaces, it can access illegal memories and cause weird behaviors.

Modifying Vector

Resizing

- Resize change the size
 - Enlarge will fill with default

```

#include <iostream>
#include <vector>

using namespace std;

void print(vector<int> v) {
    cout << "Size of V is " << v.size() << ":";
    for (int i = 0; i < v.size(); i++) cout << v[i] << ", ";
    cout << endl;
}

int main() {
    vector<int> v(3,10);
    print(v); // 10,10,10,
    v.resize(6);
    print(v); // 10,10,10,0,0,0,
    v.resize(1);
    print(v); // 10,
    v.resize(7);
    print(v); // 10,0,0,0,0,0,0
}

```

Modify

- pop_back
 - Erase last element
- insert(position,value)
- erase(position)
- Careful!

- Both insert and erase position must be valid
- If not valid, it can crash

```
#include <iostream>
#include <vector>

using namespace std;

void print(vector<int> v) {
    cout << "Size of V is " << v.size() << ": ";
    for (int i = 0; i < v.size(); i++) cout << v[i] << ", ";
    cout << endl;
}

int main() {
    vector<int> v(3,8); // v.begin() is called iterator
    v.insert( v.begin(), 1); // 1, 8, 8, 8
    v.insert( v.begin()+3, 2); // 1, 8, 8, 2, 8
    v.insert( v.end(), 3); // 1, 8, 8, 2, 8, 3
    print(v);
    v.erase(v.begin()); // 8, 8, 2, 8, 3
    v.erase(v.begin()+2); // 8, 8, 8, 3
    print(v);
    v.pop_back(); // 8, 8, 8
    print(v);
}
```

std::find & std::sort

Find

- find(a,b,c)
 - a and b are position (iterator)
 - find c from a to *before* b
 - if not found return b
- Must #include <algorithm>

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int main() {
    vector<int> v = {9,-1,3,7,5,2,1,4};

    int x = 5;
    if ( find(v.begin(), v.end(), x) != v.end() ) {
        cout << "found" << endl;
    } else {
        cout << "not found" << endl;
    }

    if (find(v.begin(), v.begin()+3, 3) != v.begin()+3) cout << "FOUND" << endl;

    // How many "YES" will be printed? (CHEAT QUESTION)
    if (find(v.begin(), v.begin()+2, x) != v.end()) cout << "YES" << endl; // YES
    if (find(v.begin(), v.begin()+4, x) != v.end()) cout << "YES" << endl; // YES
```

```

    if (find(v.begin()+4, v.begin()+2, x) != v.end()) cout << "YES" << endl; // YES
    if (find(v.begin()+4, v.begin()+8, x) != v.end()) cout << "YES" << endl; // YES
    // First 3 can't even give v.end(), last did find so it gave that position instead
}

```

Sort

```

#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

void print(vector<int> v) {
    cout << "Size of V is " << v.size() << ": ";
    for (int i = 0; i < v.size(); i++) cout << v[i] << ", ";
    cout << endl;
}

int main() {
    vector<int> v = {9,-1,3,7,5,2,1,4};

    print(v); // 9, -1, 3, 7, 5, 2, 1, 4,
    sort(v.begin()+2, v.begin()+6);
    print(v); // 9, -1, 2, 3, 5, 7, 1, 4,
}

```

Functions that Work with Vector

- sort
- find
- min_element
- max_element
- lower_bound
- upper_bound

Need to use *iterator*

Lecture 4

Vector Iterator

Iterator

- Iterator is a *pointer to position*
- First element is `begin()`
- The one *after* the last element is `end()`
- For insert, valid position is from `begin()` to `end()`, inclusive
- For erase, valid position is from `begin()` up to *before* `end()`
- Different vector, different iterator
 - `v1.end()` is not the same as `v2.end()`

Iterator Arithmetic

- We can add by integer to an iterator
- We can subtract two iterators of the same type
- We can use increment (`++`) and decrement (`--`)

```

#include <iostream>
#include <vector>
using namespace std;

```

```

int main() {
    vector<int>    v1 = {0, 10, 20, 30, 40, 50, 60, 70, 80};
    vector<float> v2 = {0.2, -4, 0.13, 3.14, 2.73};

    vector<int>::iterator it1 = v1.begin() + 3; // 30
    vector<float>::iterator it2 = v2.end();      // after 2.73

    // getting value at iterator by using "*" operator
    cout << *it1 << endl;      // 30
    cout << *(it2-1) << endl; // 2.73
    cout << *it1+2 << endl;    // 32

    // iterator arithmetics
    vector<int>::iterator it3 = it1 + 2;
    cout << "data at it3: " << *it3 << endl; // 50
    cout << "difference of it3,it1: " << (it3 - it1) << endl; // 2

    vector<float>::iterator it4 = v2.end();
    it4--;
    cout << "data of it4: " << *it4 << endl; // 2.73

    // this cannot be done
    // cout << (it2 - it1) << endl;
}

```

Iterator All Elements by Iterator

```

#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int>    v1 = {0, 10, 20, 30, 40, 50, 60, 70, 80};
    vector<float> v2 = {0.2, -4, 0.13, 3.14, 2.73};

    cout << "----v1----" << endl;
    auto it = v1.begin();
    while (it < v1.end()) { // should use != in other iterator types
        cout << it - v1.begin() << ": " << *it << endl;
        it++;
    }

    cout << "----v2----" << endl;
    for (auto it = v2.begin(); it < v2.end(); it++) {
        cout << it - v2.begin() << ": " << *it << endl;
    }
}

```

- We can compare iterator
 - Beware!! Iterator of some data structure does not support > or < comparator
 - But for vectors, it's ok
- We can use auto keyword to automatically define a type of a variable

Iterator Invalidate

Some Functions that Use Iterators

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v1 = {3,0,1,2,4,-3,9,8};
    vector<float> v2 = {10.2, -4, 0.13, 3.14, 2.73};

    auto it1 = min_element(v1.begin(),v1.end());
    auto it2 = max_element(v2.begin()+2,v2.end());

    cout << *it1 << endl; // -3
    cout << *it2 << endl; // 3.14
}

```

- min_element and max_element get the iterator of min, max element

New Idiom that Iterate All Elements

- Shorter syntax for loop
- Can use reference operator (&)

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <string>
using namespace std;

int main() {
    vector<string> v1 = {"somchai", "somying", "somsak"};

    // range-based for loop
    for (string x : v1) {
        // x is a copy of each element in v1
        cout << x << ", ";
    }

    // using reference
    // x is THE element of v1, meaning we can modify it
    for (auto &x : v1) { x.replace(0,4,"--"); }
    for (auto &x : v1) { cout << x << " "; } // --hai --ing --ak
    cout << endl;
}

```

Iterator Invalidation

```

#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v1 = {10,20};

    auto = v1.end()-1; // this points to 20
    // we resize (enlarge) v1, now [it] is invalidated
    v1.resize(10);
}

```

```
// this might not crash but it actually points to somewhere not in v1
cout << *it << endl;

// this will crash the program
// v1.insert(it,99);
for (auto &x: v1) {cout << x << " " ;}
}
```

- Some operation on the data structure *invalidate* existing iterators
 - For vector, this includes *addition* and *deletion* of element (including *resize*)

Set

- Storing distinct data of same type
 - The data type must be *comparable*, i.e., we can tell if a is more or less than b
- Somewhat slow insert
- Fast look up
- Fast erase
- Iterator starts from “minimum element” and goes in increasing value direction
 - Can be used to (somewhat) fast storing

Basic

- Notice that `s` does not include duplicate elements
- Also see that when we iterate, member is sorted
 - This is distinct characteristic of set

```
#include <iostream>
#include <set>
using namespace std;

int main() {
    set<int> s = {4,1,3,2,1,1,3,4};

    cout << "Size of s is " << s.size() << endl; // 4

    s.insert(10);
    s.insert(5);
    s.erase(3);

    cout << "member of s: "; // 1 2 4 5 10
    for (auto it = s.begin(); it != s.end(); it++) {
        cout << *it << " ";
    }
}
```

Somewhat Slow Insert, Iterate but Fast Find

- We will see this in the detail around last part of this course
 - For now, please believe that
 - If there are N elements in the set
 - Insert takes time directly proportional to $\log(N)$
 - Each `it++` or `it--` takes times directly proportional to $\log(N)$
 - Each find takes time directly proportional to $\log(N)$

Set Iterator

Demos Comparing Vector & Set

- Insert
 - Vector is faster than set

- Find random number
 - Set is much faster than vector

Set Iterator

- We cannot do `s.begin() + x`
 - Because, going to the next element (which is the *successor*) in set is not as fast as vector, c++ forbids `begin() + x`
 - alternatively, you can use `auto it = s.begin(); for (int i = 0; i < 0; i++) {it++;}`
 - We cannot *compare by > or <*
- We can still use `it++` or `it--` to go to the next or previous (*successor* or *predecessor*) or `x`

```
#include <iostream>
#include <set>
using namespace std;

int main() {
    set< pair<string,int> > s = { {"somchai",5}, {"abc",6}, {"abcd",-3}, {"somchai",-4},
    {"z",0}, {"z",-1}, {"z",9} };

    for (auto &x : s) {
        cout << x.first << "," << x.second << endl;
    }

    cout << "-- find --" << endl;
    auto it = s.find({"z",-1});
    cout << (*it).first << "," << (*it).second << endl;
    it--;
    it--;
    cout << it->first << "," << it->second << endl;
    it++;
    cout << it->first << "," << it->second << endl;
}
```

Additional Function

- `set.lower_bound`
- `set.upper_bound`
- `set.count`

std::map

Associative data structure with same property as set

Map

- Is very similar to Python's dict in usage
- Is internally implemented as a set with "pair" data type
 - *Same properties, same limitations* as set but more convenience to use as associative data structure
- Associative (mapping) between a *Key Type* and a *Mapped Type*

Basic

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    // map between "Key Type" string and "Mapped Type" int
    map<string,int> m;
    m["somchai"] = 10;
    m["somying"] = -5;
    cout << "Size = " << m.size() << endl; // 2
}
```



```
// accessing unseen key create a map with default value
cout << "m[\"xxx\"] = " << m["xxx"] << endl; // 0
// each element is a pair of key type and mapped type
for (auto it = m.begin(); it != m.end(); it++) {
    cout << it->first << " is mapped to " << it->second << endl;
} // somchai: 10, somying: -5

// this will create mapping "abc" to 0 first and then increase it
m["abc"]++;
cout << "now size = " << m.size() << endl; // 4
for (auto &x : m) {
    cout << x.first << " is mapped to " << x.second << endl;
} // abc: 1, somchai: 10, somying: -5, xxx: 0
}
```

Checking If Map Has This Key?

- Use find()

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<int,string> m;
    m[1] = "somchai";
    m[99] = "nattee";

    int k = 99;
    map<int,string>::iterator it;
    if ((it = m.find()) != m.end()) {
        cout << "Key " << it->first << " is mapped to " << it->second << endl;
    } else {
        cout << "Key " << k << " does not exist in m." << endl;
    }

    // this is not the correct way to check if key exists why??
    if (m[k] != "") {
        cout << "exists" << endl;
    } else {
        cout << "does not exists" << endl;
    }
}
```

Requirement of std::set and std::map

- Set data type and map key type must be comparable
 - We must be able to compare *order* of two elements
- Type that we can use directly
 - int, bool, float, string, double, char, ... and most of other numerical data type
 - Pair can also be used if both the types of first and second are comparable
 - Pair compare first then second

Practice Reading C++ Docs

- Both map and set has *insert* and *erase* function
- What is the return value of both function of each data structure?
 - For set<int> s, can we do s.erase(20)
 - For map<string,bool> m, can we do m.erase("Somchai")??

- If we wish to erase element from index 3 to index 4096 in a vector
 - Is there any function from vector that we can easily use?

Pair-Sum Problem

- Pair Sum
 - Given an array of integers, our task is to find whether there exists a pair of elements in the array such that their summation equal to X
- Input:
 - Array of integers (our main array); this is a large array
 - M values of X , for each value X , we have to determine if a pair whose sum equal to X exists.
- Output:
 - For each value of X , print "YES" if we found such pair; print "NO" otherwise

Answer

```
#include <iostream>
#include <array>
#include <algorithm>
#include <cmath>

using namespace std;

bool check(int a[], int n, int X);

int main() {
    int a[15] = {323, 232, 138, 230, 437, 176, 127, 111, 329, 1, 296, 12, 3, 2, 6};
    int n = sizeof(a) / sizeof(a[0]);
    int X = 10000;

    cout << check(a,n,X);
}

bool check(int a[], int n, int X) {
    for (int i=0; i<(pow(2,n)); i++) {
        int t = X;
        for (int j=0; j<n; j++) {
            if (i / ((int) pow(2,j+1)) % 2 == 0) {
                t -= a[j];
            }

            if (0 == t) {
                cout << "YES" << endl;
                return true;
            }
            if (0 > t) {break;}
        }
    }
    cout << "NO" << endl;
    return false;
} // My stupid solution where I brute force every possible combinations
```

Lecture 5

Stack

การเพิ่ม / ลบข้อมูลในกองซ้อน

- ข้อมูล เข้าหลัง ออกก่อน (Last_In First-Out)

กองซ้อน : **stack**

```
bool        empty();
unsigned    size();
T           top();           // ดูบนสุด
void        push(T element); // ใส่ข้อมูล
void        pop();           // เอาออก
```

ข้อควรระวัง

- ใช้ `#include <stack>`
- `top()` ตอนไม่มีข้อมูล → พัง
- `pop()` ตอนไม่มีข้อมูล → พัง
- `stack` ไม่มี iterator
 - ดูได้เฉพาะข้อมูลที่เพิ่งใส่เข้าไปล่าสุด (`top`)
 - ไม่มี `begin()`, `end()` ให้ใช้
 - ถ้าอยากมาดูข้อมูลทุกตัวใน `stack` ต้องจำใจ `pop` ข้อมูลออกมา

ตัวอย่างการใช้งาน **Stack**

- การตรวจสอบโครงสร้างแบบซ้อนกัน เช่น การใส่วงเล็บ `(){}[]...`
- การจัดเก็บตัวแปรและการทำ `function calls`
- การประมวลผลนิพจน์ทางคณิตศาสตร์ `postFix` → Discrete Math
- การทำ `undo/redo`
- การค้นคำตอบแบบ `depth-first search`
- ...

Parentheses Checking

การตรวจสอบการใส่วงเล็บ

- ถูก : `( { ( ) [ { } ] } )`
- ผิด : เปิดปิดไม่ตรงกัน `( { ] )`
- ผิด : มีปิดมากเกินไป `( { ( ) } ) ) }`
- ผิด : มีเปิดมากเกินไป `( { ( ) }`
- วิธีทำ
 - อ่านมาทีละตัว
 - ถ้าเป็นวงเล็บเปิด ให้ `push` ลง `stack`
 - ถ้าเป็นวงเล็บปิด ให้ `pop` จาก `stack` มาตรวจสอบว่าเป็นวงเล็บเปิดที่ตรงกันกับวงเล็บปิดหรือไม่
 - เมื่อใดที่อยาก `pop` ถ้า `isEmpty` แสดงว่า ปิดมีมากเกินไป
 - เมื่ออ่านเสร็จหมด `stack` ยังมีข้อมูล แสดงว่า เปิดมีมากเกินไป

Function Call Stack

Using Stack in Java Virtual Machine

- jvm uses java stack to store
 - state of method calls
 - parameters and local variables
 - temporary memory for computation (operand stack)

```
public class StackFrame {
    public static void main(String[] args) {
        a(3,2);
        b(5);
    }
    static void a(int x, int y) {
        int z = x/y;
        b(z);
    }
}
```

```

    }
    static void b(int x) {
        ++x;
    }
}

```

We make a stack frame of function calls, each element consists of arguments and returning argument of the function. Function main contains args and RA JVM and calls a, a will be added to stack and contains z, y, x, and RA 04:, then it calls b. After b is finished, it will come back to line 9 and popped itself. It will do a and go to line 4, and so on. Arguments in each functions are separated variables.

```

#include <iostream>

using namespace std;

void test(int x) {
    if (x > 0) {
        int y = x-1;
        cout << "I am test(" << x << "). continue... " << endl;
        test(y);
        cout << "now x is " << x << endl;
    } else {
        cout << "I am test(0). stop"
    }
}

int main() {
    test(4);
    return 0;
}

```

Output

```

I am test(4). continue...
I am test(3). continue...
I am test(2). continue...
I am test(1). continue...
I am test(0). stop
now x is 1
now x is 2
now x is 3
now x is 4

```

Postfix Evaluation

Infix and Postfix Expressions

- infix (เติมกลาง)
 - $a + b * c / d - 2, (a + b) * c / (d - 2)$
 - use order of operations
 - use parentheses
- postfix (เติมหลัง)
 - $a b c * d / + 2 -, a b + c * d 2 - /$
 - order is from left to right
 - parentheses may be omitted

```

1   2   +   3   *   4   1   -   /
      3   3   *   4   1   -   /
          9   4   1   -   /

```

9 3 /
 3

Calculate Postfix Expressions with Stack

- Value of $2\ 3\ +\ 4\ 5\ -\ 6\ *\ +\ ?$
- Use stack to help
- Solution
 - Check each element in postfix from left to right
 - If it is operand, push stack
 - If it is operator, pop elements from stack as many as operator needs, then push the result
 - After complete, answer will be at the top of stack

Convert Infix to Postfix

Use Stack to Convert

- input : infix expression
- output : postfix expression
- Steps
 - Read each element in infix
 - If it is operand, move it to the back of output
 - If it is operator
 - May pop operator to the end of output
 - push operator to stack
 - If all infix is read
 - pop all operators to the end of input

```
string infix2postfix(string &infix) {
    int n = infix.length();
    string postfix = "";
    stack<char> s;
    for (int i=0; i<n; i++) { // Read each element
        char token = infix[i];
        if (token == operand) {
            postfix += token; // if operand, add to output
        } else {
            while ( ( ??? ) ) {
                postfix += s.top(); // if operator, pop stack to result,
                s.pop();
            }
            s.push(token); // then push new operator
        }
    }
    while (!s.empty()) {postfix += s.top(); s.pop();}
    return postfix;
}
```

Priority of Operators

input 2 * 3 + 4

read	output	stack	
2			
*	2		
3	2	*	
+	2 3	*	(* comes before and is more important than +)
4	2 3 *	+	
	2 3 * 4	+	
	2 3 * 4 +		

input 2 + 3 * 4

read	output	stack
2		
+	2	
3	2	+
*	2 3	+
	2 3	* +
	2 3 *	+
	2 3 * +	

Code to Compare Priority

```
string infix2postfix(string &infix) {
    int n = infix.length();
    string postfix = "";
    stack<char> s;
    for (int i=0; i<n; i++) {
        char token = infix[i];
        if (priority.find(token) != priority.end()) {
            postfix += token;
        } else {
            int p = priority[token]; // Get priority of the new operator
            while ( !s.empty() && priority[s.top()] >= p ) { // lazy evaluation; left first
                postfix += s.top(); // pop if top operator is not less important than new op
                s.pop();
            }
            s.push(token);
        }
    }
    while (!s.empty()) {postfix += s.top(); s.pop();}
    return postfix;
}
```

Lazy Evaluation

Setting Priority of Operators

- ^ represents power 7
- ^ before + - * /
- * / before + -
- * same as / 5
- + same as - 3

```
map<char,int> priority =
    {{'+',3},{'-',3},{ '*',5},{ '/',5},{ '^',7}};
```

Complications with Parentheses

- In parentheses is like sub-expression
- Open parentheses always push (very high priority)
- Close parentheses is very low priority
- If found, pop until sees open parentheses

input 2 * (3 + 4)

read	output	stack
2		
*	2	
(	2	*

```

3          2          ( *
+          2 3        ( *
4          2 3        + ( *
)          2 3 4      + ( * (pop all)
          2 3 4 + *

int p = priority[token];
while ( !s.empty() && priority[s.top()] >= p ) {
    postfix += s.top();
    s.pop();
}
s.push(token);

int p = outpriority[token];
while ( !s.empty() && inpriority[s.top()] >= p ) {
    postfix += s.top();
    s.pop();
}
if (token == ')') s.pop(); else s.push(token);

```

Priority of Operators with Parentheses

	+	-	*	/	^	(	)
In expression	3	3	5	5	7	9	1
In stack	3	3	5	5	7	0	

Right to Left Operator

	+	-	*	/	^	(	)
In expression	2	2	4	4	8	9	1
In stack	3	3	5	5	7	0	

Note Power in expression is more important than in stack

input 2 + 3 - 4 ^ 5 ^ 6

read	output	stack
2		
+	2	
3	2	+
-	2 3	+
4	2 3 +	-
^	2 3 + 4	^ -
5	2 3 + 4	^ -
^	2 3 + 4 5	^ -
6	2 3 + 4 5	^ ^ -
	2 3 + 4 5 6	^ ^ -
	2 3 + 4 5 6 ^ ^ -	

Conclusion

- Implementations of stack to solve problems
- Main operations: push / pop / top
- Make stack with array
- If reserve enough space, all runs is $\Theta(1)$; constant time

Lecture 6

Queue

- Just like a normal queue in real life.
 - First-in-First-Out data structure
 - One way in (back of queue), one way out (front of queue)
 - push = add data to the back of the queue
 - pop = remove data from the head of the queue

Basic

```
#include <iostream>
#include <queue>
#include <vector>

using namespace std;

int main() {
    queue<int> q;
    q.push(1);
    q.push(2);
    q.push(3);
    while (q.empty() == false) {
        cout << q.front() << endl;
        q.pop();
    }
    cout << "-- example 2 --" << endl;
    queue<vector<int>>> q2;
    vector<int> v1 = {1,2,3};
    vector<int> v2 = {99,88,-1};
    q2.push( v1 );
    q2.push( v2 );
    cout << q2.back()[1] << endl;
    cout << q2.front().size() << endl;
    auto x = q2.front();
    q2.pop();
    cout << x[0] << endl;
}

size_t    q.size()
bool      q.empty()
void      q.push(T data)
void      q.pop()
T         q.front()
T         q.back()
```

Limitation

- Same limitation as stack
 - No iterator
 - No begin(), end()
 - Can only access front and back of the queue
 - If we wish to access all members, we have to pop it all
 - Do not call front(), back(), pop() when the queue is empty

Radix Sort

Queue Application: Fast sorting with no comparison

Overview

- Put all data in an array
- For each digit X , from LSD to MSD
 - PUT TO QUEUE step: Sort by digit X by putting all of data from the array into B queues
 - B is the base of the number
 - For example, for a base 10 number, we will have Queue[0] to Queue[9]
 - Put into the queue labelled with that digit
 - GET FROM QUEUE step: Start from queue 0 to queue $B - 1$, remove data from the queue and put back to the array

Example

Input 115 15 42 305 21 8 403

Round 1 (digit 0), from queue

(0)	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)
					305				
					15				
	21	42	463		115			8	

⇒ 21 42 463 115 15 305 8

Data is now sorted by the last digits, because we pop from queue 0 to 9

Round 2 (digit 1), from queue

(0)	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)
8	15								
305	115	21		42		463			

⇒ 305 8 115 15 21 42 463

Data is now sorted by last two digits, because we goes from queue 0 to 9, which is grouped by digit 1 and the data in each queue is sorted by the last digit.

Round 3 (digit 2), from queue

(0)	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)
42									
21									
15									
8	115		305	463					

⇒ 8 15 21 42 115 305 463

Code

```
#define base 10

int getDigit(int v, int k) {
    // return the kth digit of v (MSD is digit 0)
    int i;
    for (int i=0; i<k; i++) v /= base;
    return v % base;
}

// d = number of digits
void radixSort(vector<int> &data, int d) {
    queue<int> q[base];
    for (int k=0; k<d; k++) {
        for (auto &x : data) {
            q[getDigit(x,k)].push(x);
            for (int i=0, j=0; i<base; i++) {
```

```

    while(!q[i].empty()) {
        data[j++] = q[i].front(); q[i].pop();
    }
}
}
}
}
}

```

Breadth First Search

Queue Application: Gotta generate 'em all

The Problem

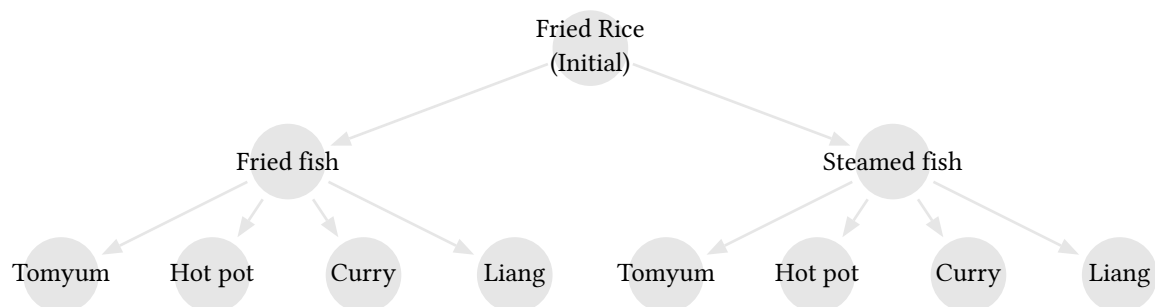
- Given a positive integer X
- Start with a number 1, find a sequence of arithmetics operations, either “ $\times 3$ ”, or “ $/ 2$ ” that makes 1 into X
 - the $/ 2$ is integer division, e.g., $5 / 3 = 1$ (not 1.6666)
 - The sequence must be as short as possible
- Example
 - $10 = 1 \times 3 \times 3 \times 3 \times 3 / 2 / 2 / 2$
 - $31 = 1 \times 3 \times 3 \times 3 \times 3 \times 3 \times 3 / 2 / 2 / 2 / 2 / 2 \times 3 \times 3 / 2$

The Idea

- Generate all possible sequences
 - Start with length 1, 2, 3, ... until we find one that gives X
- This is called an *exhaustive search* algorithm
 - Systematically enumerate all possible somethings

Tree Structure (Search Tree)

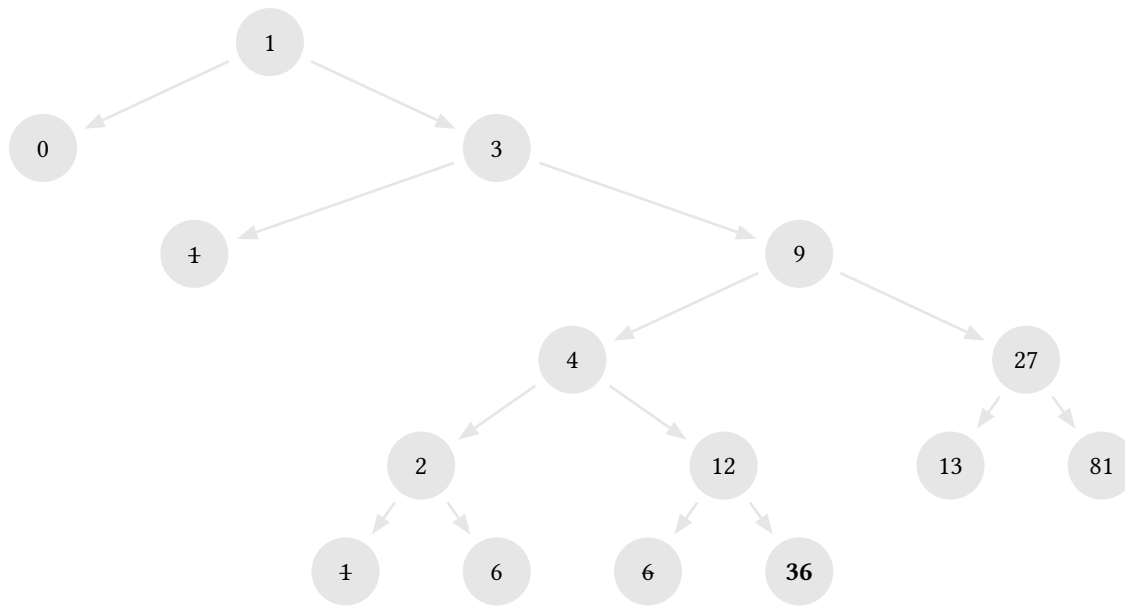
- A structure to illustrate *search* algorithm
- Divide into steps
 - Start with *intial solution*
 - For each possible outcome (called a *state*) of each step, *generate all* proper possible *next step*
 - Also, check if the current step is what we needed
- Written as a diagram of *node* and *edge*



M3D2 Problem & Solution

Back to Our Problem

- Start with 1
- Each step is either $\times 3$ or $/ 2$
- Issue: might get repeated number
 - Solution: if we have found it, do not generate new step



M3D2 Code

Code

- Queue makes ordering of how we pick a state to enumerate
- From top to bottom and left to right

```

void m3d2(int target) {
    map<int, int> prev;
    queue<int> q;
    int v = 1;
    q.push(1); prev[1] = -1;
    while (!q.empty()) {
        v = q.front(); q.pop();
        if (v == target) break;
        int v2 = v/2;
        int v3 = v*3;
        if (prev[v2] == 0) {q.push(v2); prev[v2] = v;}
        if (prev[v3] == 0) {q.push(v3); prev[v3] = v;}
    }
    if (v == target) showSolution(v, prev);
}

```

Display Solution

x	0	1	2	3	4	6	9	12	13	27	36	39	40
prev[x]	1	-1	4	1	9	2	3	4	27	9	12	13	81

showSolution

```

void showSolution(int v, map<int,int>& prev) {
    string out = "";
    while (prev[v] != -1) {
        if (prev[v] * 3 == v) {
            out = "x3" + out;
        } else {
            out = "/" + out;
        }
        v = prev[v];
    }
}

```

```

    }
    out = "1" + out;
    cout << out << endl;
}

```

$\Rightarrow 36 = 1 * 3 * 3 / 2 * 3 * 3$

Lecture 7

Priority Queue

Queue with privilege

Intro

- Priority Queue is ...
 - A queue with priority
 - Item with high priority is promoted to the front of the queue
 - There is *no back of the queue*
 - Priority is defined by having more value
 - Comparison, by default, is to use operator `<`, i.e., if item `A < B` is true, then `B` has *higher priority*
 - We can have custom comparator
- Has the *same interface* as stack

Example

For intuitive purpose only! This is not really how priority_queue work internally

```

push(10)
< 10      >

```

```

push(30)
< 30 10   >

```

```

push(20)
< 30 20 10 >

```

```

pop()
< 20 10   >

```

```

push(15)
< 20 15 10 >
|
top

```

Basic

```

size_t      q.size()
bool        q.empty()
void        q.push()
void        q.pop()
T           q.top()
* there is no q.back()

```

Limitation

- Same limitation as stack, queue
 - *no iterator*
 - No `begin()`, `end()`
 - Use `#include <queue>`
 - Can only access `top()` of the queue
 - If we wish to access all members, we have to pop it all

- Do not call `top()`, `pop()` when the queue is empty
- The data type must be *comparable* (similar to set and map)

Class

Quick Summary

- Syntax
 - class declaration must end with ;
 - Function definition can be outside of the class
 - Access modifier is `public:`, `private:`, `protected:`
 - Constructor is a function with the same name of the class with no return type
- Object is a variable (instantiation) of a class
 - When declared, a constructor is called

Example 1

```
#include <iostream>
#include <string>
using namespace std;

class Student {
public:
    void setFullName(string name, string surname) {
        this->name = name;
        this->surname = surname;
    }
    string getFullName() {
        return "[" + name + " " + surname + "];"
    }
private:
    string name, surname;
};

int main() {
    Student a;
    Student b;
    a.setFullName("nattee", "niparnan");
    cout << a.getFullName() << endl;
    cout << b.getFullName() << endl;
}
```

Declaring class before writing functions

```
class Student {
public:
    void setFullName(string name, string surname);
    string getFullName();
private:
    string name,surname;
};

void Student::setFullName(string name, string surname) {
    this->name = name;
    this->surname = surname;
}

string Student::getFullName() {
```

```
    return "[" + name + " " + surname + "];"
}
```

Output

```
[nattee niparnan]
[ ]
```

Example 2: Constructor

```
#include <iostream>
#include <string>
using namespace std;

class Student {
public:
    Student(float score) {gpax = score;}
    void setFullName(string name, string surname) {
        this->name = name;
        this->surname = surname;
    }
    string getFullName() {return "[" + name + " " + surname + "];"}
    bool is1stHonor() {return gpax >= 3.6;}
private:
    string name, surname;
    float gpax;
};

int main() {
    Student a(2.95);
    a.setFullName("nattee", "niparnan");

    cout << a.getFullName() << endl;
    if (a.is1stHonor()) {cout << "YES" << endl;} else {cout << "NO" << endl;}
    // Student b; // <-- cannot compile beacuse there is no default constructor
}
```

Operator Overloading

How C++ has a function for each operator

Overview

- Let say we write `a + b` when `a` and `b` is an object of some classes
 - This can be considered the same as calling a function `plus(a,b)`
 - C++ allows us to write a function for many operator and use it as an operator
 - For example we can write a function `times(a,b)` and let it be used as `a * b`
- This is what we called operator overloading

Instead of writing

```
Matrix multiply(Matrix a1, Matrix a2) {
    multiply(a, multiply(b,c))
}
```

We want to write

```
Matrix a, b, c
a * (b * c)
```

Example

```
#include <queue>
#include <iostream>
```

```
#include <string>

using namespace std;

// use operator + symbol that will be overloaded
string operator*(string & lhs, const int & rhs) {
    string result = "";
    for (int i=0; i<rhs; i++) {
        result = result + lhs;
    }
    return result;
}

int main() {
    string a = "abc ";
    cout << (a * 3) << endl; // this gives "abc abc abc "
}
```

- Function must be named operator followed by the operator that we will overload
- Some operator takes two parameters (such as +, -, *, /, %)
- Some takes one (such as ++, --, !, *, &)

Overloading less-than operator

Using with data structure that require sorting

- We have seen several data structure that requires comparability of the data, such as set, map and priority queue
- If we want to use our class with these data structure, we need to tell them how can we use them
- There are multiple ways to achieve this
 - Let us consider operator overloading

Overloading <

- As stated earlier, set, map, and priority_queue use *operator <* to compare two elements
- It does not work if we overload *operator >*

```
class Student {
public:
    Student(float score, string a, string b) {
        name = a;
        surname = b;
        gpax = score;
    }
    bool is1stHonor() {return gpax >= 3.6;}
    // not good, now our data is public
    string name, surname;
    float gpax;
    // overloading <
    bool operator<(const Student & other) const {
        return gpax < other.gpax;
    }
};

int main() {
    Student a(2.95, "nattee", "niparnan");
    Student b(4.00, "attawith", "sudsang");
    cout << (a < b) << endl; // 1
}
```

```

priority_queue<Student> pq;
pq.push(a);
pq.push(b);
cout << pq.top().name << endl; // attawith
}

```

Custom Comparator

Why custom?

- By overloading *operator <*, we have defined default ordering of that class
- What if we need another ordering, just for this *priority_queue* only
 - For example, *Student* is ordered by *gpax* by default
 - What if we want our *priority_queue* to order by name instead, while keeping the *Student* default ordering elsewhere
 - Better, can we have multiple *priority_queue* with different ordering?
- Can be done via comparator class

Example

```

#include <iostream>
#include <string>
#include <queue>
using namespace std;

class Student {...};

// Comparator class use () overloading
class StudentByNameComparator {
public:
    bool operator()(const Student& lhs,
                    const Student& rhs) {
        return lhs.name < rhs.name;
    }
};

class GpaxThenName {
public:
    bool operator()(const Student& lhs,
                    const Student& rhs) {
        if (lhs.gpax == rhs.gpax)
            return lhs.name < rhs.name;
        return lhs.gpax < rhs.gpax;
    }
};

int main() {
    Student a(2.95, "nattee", "niparnan");
    Student b(4.00, "attawith", "sudsang");
    Student c(4.00, "vishnu", "kotrajaras");
    cout << (a < b) << endl; // 1
    StudentByNameComparator comp1;
    GpaxThenName comp2;

    cout << comp1(a,b) << endl; // 0; can use like function

    // Use 3 template parameters
    priority_queue<Student,                // class to compare

```



```

        vector<Student>, // vector of that class
        StudentByNameComparator> pq(comp1); // comparator class

pq.push(a);
pq.push(b);
cout << pq.top().name << endl; // nattee

priority_queue<Student,
               vector<Student>,
               GpaxThenName> pq2(comp2);

pq2.push(a);
pq2.push(b);
pq2.push(c);
cout << pq2.top().name << endl; // vishnu
}

```

Another Method, lambda-function

```

#include <iostream>
#include <string>
#include <queue>
using namespace std;

int main() {
    auto compare = [](const string& lhs, const string& rhs) {
        return lhs.size() < rhs.size();
    };

    cout << "Result of compare function = " << compare("xxx","z") << endl;

    priority_queue<string,vector<string>,decltype(compare)> pq(compare);
    pq.push("somchai");
    pq.push("z");
    pq.push("abc");
    while (pq.empty() == false) {
        cout << pq.top() << endl;
        pq.pop();
    }
}

```

Output

```

Result of compare function = 0
somchai
abc
z

```

Priority Queue Template

Templating of priority_queue

- priority_queue requires 3 template parameters
- priority_queue<T, Container = vector<T>, Compare = less<T>>
- The first one is required (which is the type of the data)
- The second and the third is optional (it has default type)
 - Second is the container (for now, just don't think about it)
 - Third is the class for comparator (the class that we use to compare)
 - This one is default to less<T>

```

#include <iostream>
#include <string>

```

```
#include <queue>
using namespace std;

int main() {
    less<int> x;    // comparator that checks a < b
    greater<int> y; // checks a > b

    int a = 10;
    int b = 3;
    cout << x(a,b) << endl;    // 0
    cout << y(a,b) << endl;    // 1
}
```

Using Comparator for set and map

- To use custom class with set and map, we need to do the same thing, let set and map know how to sort the data
 - Either make default ordering (overload <) in the custom class
 - Or use custom comparator when declare
- For set, the declaration is `set<T, Compare = less<T>>`
- For map, the declaration is `map<Key, T, Compare = less<Key>>`

Assignment

- Is any of `vector<int>`, `set<int>`, `map<int, string>`, `queue<bool>`, `stack<vector<int>>` comparable?
 - For any class that is "YES", how it is ordered?
 - For example, if `vector<int>` is comparable, how `{1,2,3}` is compared to `{1,2,3,4}` or `{2,3,4}`

Answer

- `vector<int>` compare each element from the left; `{2,3,4} > {1,2,3,4} > {1,2,3}`
- `set<int>` compare the maximum element () of each set, if equal then look the next max
- `map<int, string>` compare by Key set, then compare max of value set
- `queue<bool>` is not comparable
- `stack<vector<int>>` is not comparable

Summary

Data Structure Summary

Data Structure	Pro	Cons	Remark
<code>pair<T1, T2></code>	Nothings... It just a pair of two data type		
<code>vector<T></code>	- Fast access [] - Fast append $\mathcal{O}(1)$	- Slow find $\mathcal{O}(N)$ - Slow insert, Slow Erase $\mathcal{O}(N)$	
<code>set<T></code>	- Fast find $\mathcal{O}(\log(N))$ - Item is sorted	- Slower to just append data than vector, stack, queue $\mathcal{O}(\log(N))$ - Iterate is also slow - Takes lots of memory $\mathcal{O}(\log(N))$	Require comparator
<code>map<Key, T></code>			- Associative data type - Also require comparator of Key_Type
<code>stack<T></code>	Very fast push, pop $\mathcal{O}(1)$	Very limited functionality but has special uses	- No iterator - Order of data coming out depends on something (stack, queue depends on WHEN it is pushed, pq depends on value) - pq requires comparator
<code>priority_queue<T></code>			

More Data Structure

- C++ has more data structure not really covered right now
 - `list` is a vector with faster input / `erase` but does not have fast access
 - `unordered_set`, `unordered_map` are set and map that the data is not sorted but is much faster
 - `deque` is a queue that can push, pop at both ends
 - `multiset`, `multimap` are set and map that allows duplicated entries

Lecture 8

Create Your Own CP::pair

Intro

- We start writing our own data structure
 - In our own *namespace* (CP)
- We start with *pair*
 - Good introduction on writing a class
 - Some C++ feature we need to use
 - `const`
 - pass-by-value, pass-by-ref
 - Header file

What should we write in a class

- Class name and namespace
- Member *variables* which define what data we want to store in the class
- Member *functions* which define behavior of the class and how member variable is manipulated
 - Include lots of special function such as *constructor*, *destructor*, *overloading*

CP::pair version 0.1 (minimal version)

- Templating
 - Parameter of a code
- No member functions
 - When we declare a pair, T1 and T2 must be declared as a type

```
namespace CP {
    template <typename T1, typename T2>
    class pair{
    public:
        T1 first;
        T2 second;
    };
}

int main() {
    CP::pair<int, string> p1; // In p1, T1 is int, T2 is string
    CP::pair<bool, int> p2;  // In p2, T1 is bool, T2 is int
}
```

Capabilities

- Can hold 2 pieces of data
- Can use template
- Works with operator =
- Has copy constructor
- Cannot check equal
- Cannot check less than

```

#include <iostream>
#include <string>
using namespace std;
namespace CP {
    template <typename T1, typename T2>
    class pair{
    public:
        T1 first;
        T2 second;
    };
}

int main() {
    CP::pair<int,string> p1, p2; // default ctor
    p1.first = 20; p1.second = "somchai";
    CP::pair<int,string> a(p1); // copy ctor
    p2 = p1;

    cout << p2.first << "," << p2.second << endl;
}

/*
if (p1 == p2) { // won't compile
    cout << "yes" << endl;
}

if (p1 < a) { // won't compile
    cout << "yes" << endl;
}
*/

```

CP::pair version 0.2 (comparator overload)

```

#include <iostream>
#include <string>
#include <set>
using namespace std;
namespace CP {
    template <typename T1, typename T2>
    class pair{
    public:
        T1 first;
        T2 second;

        // operator
        bool operator==(const pair<T1,T2> &other) {
            return (first == other.first && second == other.second);
        }

        bool operator<(const pair<T1,T2>& other) const { // operator< must be const
            return ((first < other.first) ||
                    (first == other.first && second < other.second));
        }
    };
}

int main() {

```

```

CP::pair<int,string> p1, p2; // default ctor
p1.first = 20; p1.second = "somchai";
CP::pair<int,string> a(p1); // copy ctor
p2 = p1;
cout << p2.first << "," << p2.second << endl; // 20, somchai

if (p1 == p2) {cout << "yes" << endl;} // yes
if (p1 < a) {cout << "yes" << endl;}
set<CP::pair<int,int>> s;
s.insert({1,2});
// Now we can compare and use less than (and also set, map, priority queue)
cout << s.begin()->second << endl; // 2
}

```

Const Member

Const Parameter in Function

```

void test(int &x, const int& y) {
    x++; // Okay; doable
    cout << y << endl; // Okay, we only read y
    for (int i=0; i<y; i++) { // Okay, too
        cout << i << endl;
    }
    y--; // ERROR: we promised NOT to change y
}

```

- Declared by putting a keyword const as a prefix
- Const parameter must not be modified inside the function
- Why? So that we know that the function does not modify the data
 - Especially when we pass-by-reference

Difference of Pass-by-reference && Pass-by-value

Pass-by-value

- The *arguments* can be either constants or variables
- Modifying *parameters* (variable inside the function) does not change the *argument's* value
- The *argument's* value is copied to the parameter
 - **SLOWER!!!** Because we have to copy

Pass-by-reference

- The *arguments* must be variables
- The *parameters* are the argument's variables
 - Modify the *parameter* also modify the *argument's* variable
- **Faster**

Const Member Function

```

class ccc {
public:
    int a,b;

    void inspect() const { // This function promises NOT to change anything
        if (a < b) cout << "yes" << endl; // Okay

        // b += 20; // <-- NOT OKAY
    }

    void mutate() { // This function might change something
        if (a < b) a += 10; // Okay
    }
}

```

```

    }
};

void test2(ccc& changeable, const ccc& unchangeable) {
    changeable.inspect();
    changeable.mutate();
    unchangeable.inspect();
    unchangeable.mutate(); // ERROR: attempt to change unchangeable object
}

```

- Declared by putting a keyword `const` after the function declaration
- Const member function cannot modify any member data
 - Also cannot call any other function that is not const
- Why? So that we know that the function does not modify its data

Constructor & List Initialization

Custom Constructor

- In STL spec of `std::pair`, it has custom constructor called *initialization constructor*

```

namespace CP {
    template <typename T1, typename T2>
    class pair{
    public:
        T1 first;
        T2 second;

        // Custom constructor, using initialization list
        pair(const T1 &a, const T2 &b) {
            first = a;
            second = b;
        }

        // When we have a constructor, a default constructor is not auto-generated

        // Operator
        bool operator==(const pair<T1,T2> &other) {...}
        bool operator<(const pair<T1,T2>& other) {...}
    };
}

int main() {
    CP::pair<int, bool> p(10,false);
    CP::pair<string, int> q("abc",42), r("",0);

    cout << (q < r) << endl; // 0
    priority_queue<CP::pair<string,int>> pq;
    pq.push(r);
    pq.push(q);
    cout << pq.top().first << endl; // abc

    CP::pair<string, int> x(q);
    CP::pair<string, int> y = x;

    // All below cannot be compiled
    // CP::pair<string, int> w;
    // vector<CP::pair<int, int>> v(10);
}

```

Initialization List

- Instead of writing a code to assign a value to each member, we can use *initialization list*
 - A little bit shorter code
 - Also little bit faster
 - Only way to init *const* member

```
namespace CP {
    template <typename T1, typename T2>
    class pair {
    public:
        T1 first;
        T2 second;

        // custom constructor, using initialization list
        pair(const T1 &a, const T2 &b) : first(a), second(b) { }
    };
}
```

Default Constructor

- A constructor used when we simply declare an object
- Auto-generated by init all members with its default constructor
- Won't be auto-gen if we have any other constructor

```
namespace CP {
    template <typename T1, typename T2>
    class pair {
        T1 first;
        T2 second;

        // Constructor
        pair() : first(), second() { }
        pair(const T1 &a, const T2 &b) : first(a), second(b) { }
    };
}
```

Include & Header file

Include File

- Usually, our data structure (which is written as a class) will be used by several programs in several files.
 - It is better NOT TO copy our data structure code into each each file
- Rather, put our code in a file and include it where it is needed
 - Better if we want to change it
 - Better compilation
- Introducing ".h" files

C++ header file (.h) and #include

- To put a content of one file into another file, we use `#include "filename"` keyword
- C++ will simply put the content of *filename* into where we `#include` it
- `#include` has more benefit
 - Separation of *declaration* (what it is) and *definition* (how it works)
 - Not really explored in this class
- We usually use .h for a file that we will include but that is not a rule, we can use other extension

Problem with include

a.h

main.cpp (expanded)

```
class a {
    int m1, m2;
```

```

class a {
    int m1, m2;
};

b.h

#include "a.h"

class b {
    a x;
};

c.h

#include "a.h"

class c {
    a y;
};

main.cpp

#include "b.h"
#include "c.h"

int main() {
    b b1;
    c c1;
}

```

Fix problem

```

a.h

#ifndef A_H
#define A_H
class a {
    int m1, m2;
};
#endif

b.h

#include "a.h"

class b {
    a x;
};

c.h

#include "a.h"

class c {
    a y;
};

```

```

};

class b {
    a x;
};

// There are multiple copies of class a
class a {
    int m1, m2;
};

class c {
    a y;
};

int main() {
    b b1;
    c c1;
}

```

main.cpp (expanded)

```

#ifndef A_H
#define A_H
class a {
    int m1, m2;
};
#endif

class b {
    a x;
};

// class a here is taken out of
// compilation because A_H is defined
#ifndef A_H
// #define A_H
// class a {
//     int m1, m2;
// };
#endif

class c {
    a y;
};

int main() {
    b b1;
}

```



```

        c c1;
    }

```

Summary

Major

- **Templating**: allow user to write a code that works with different data type
- **Constructor**: a function called when we create a variable
 - Default constructor
- **Operator overloading**: for less-than and equality
- pass-by-ref vs pass-by-value

Minor

- Header file
- const
- List initialization

Exercise (no grader)

1. Modify operator< so that it compare *second* before *first*
2. Modify operator< so that when we call sort(v.begin(), v.end()) where w is a vector of our pair, it is sorted from *Max* to *Min*
3. Write operator!= and operator>=

Final version CP.h

```

#ifndef _CP_PAIR_INCLUDED_
#define _CP_PAIR_INCLUDED_

namespace CP {
    template <typename T1, typename T2>
    class pair {
    public:
        T1 first;
        T2 second;

        pair() : first(), second() {}

        pair(const T1 &a, const T2 &b) : first(a), second(b) {}

        bool operator==(const pair<T1,T2> &other) {
            return (first == other.first && second == other.second);
        }

        // 1.
        bool operator<(const pair<T1,T2> &other) const {
            return ((second < other.second) ||
                    (second == other.second && first < other.first));
        }

        // 2.
        bool operator>(const pair<T1,T2> &other) const {
            return ((first > other.first) ||
                    (first == other.first && second > other.second));
        }

        // 3.
        bool operator!=(const pair<T1,T2> &other) const {
            return !(*this == other);
        }
    };
}

```

```

    }

    bool operator>=(const pair<T1,T2>& other) const {
        return !(*this < other);
    }

};
}
#endif

```

Lecture 9

CP::vector

Our first “real” data structure

Intro

- Now we will create more complex data structure CP::vector
- It can store variable length array
 - Implemented as a dynamic array
- Can be accessed by operator []
 - Additional operator to be overloaded
- We also have to create our own iterator
 - Implemented as a pointer

Key Idea

- Vector stored 3 things (3 member data)
 - **mData**: A dynamic array, large enough space to store current data and might have reserved space
 - **mSize**: Number of data stored
 - **mCap**: Size of the dynamic array (maybe large than **mSize**)
- If the dynamic array is full and more data is being added, we create a new dynamic array and relocate data to the new array
 - This is called **expand**
 - Each **expansion** takes very long time
- Dilemma
 - Large reserve = less often relocation but use more memory
 - Small reserve = more frequent relocation but less memory

Example

```

mSize  mCap      mData
  4      5    [10, 3, 1, 5, ]

```

push_back(9)

```

  5      5    [10, 3, 1, 5, 9]

```

push_back(4)

```

  6     10    [10, 3, 1, 5, 9, 4, , , , ]

```

How much reserve should we have?

- Whenever we need to relocate, we **double** the capacity that we currently have
- If we start with a vector with zero size and continuously add data one by one, for example by push_back
 - Each expansion will take time **equal** to **the number of data** when we relocate (this is very slow)

- By doubling the size every time we expand, we can show that on average, **each addition of a data** (such as push_back, insert) **takes constant time!!!**

Pointer

How to implement a dynamic array (and also iterator)

Pointer & Memory

- Each variable is some block in computer memory
 - Programming language just map our **variable name** to that **block of memory**
 - Programming language works with the address of that block
- Pointer variable is a variable that stor **address of memory**
 - Pointer needs type i.e., address of int is not the same as address of bool
 - We can use operator & to ask for the **address** of a **variable**
 - We can use operator * to ask for the **data** of an **address**

Example

```
int x, y; // this is normal variable x and y
int *a; // this is int pointer variable a
int *b; // another int pointer

x = 10; // say x is at address 3267
a = &x; // a points to x
b = a; // b also points to x
*b = 30; // change the address value to 30
y = *b; // set value of y to 30 (hard copy)

cout << &x << endl; // 0x7ffee22885ac
cout << &y << endl; // 0x7ffee22885a8 +4
cout << &a << endl; // 0x7ffee22885a0 +8
cout << &b << endl; // 0x7ffee2288598 +8
cout << sizeof(int) << endl; // 4 bytes
cout << sizeof(*int) << endl; // 8 bytes
```

Pointer Arithmetic

- Pointer can be added, subtracted by integer
 - It **moves the address by the size of the type** of the pointer
 - For example, when X is an int (which is 4 bytes) X + 10 result in an address 40 bytes away from X
- Two pointers of the same type can be subtracted
 - The result is the address **difference divided by size of the type** of the pointer

```
int main() {
    bool x, y, z;
    x = 1; y = 2; z = 3;
    bool *a, *b;

    a = &x;
    b = a+2;

    cout << "&x = " << &x << endl; // &x = 0x6afed4
    cout << "&y = " << &y << endl; // &y = 0x6afed8
    cout << "&z = " << &z << endl; // &z = 0x6afedc
    cout << " a = " << a << endl; // a = 0x6afed4
    cout << " b = " << b << endl; // b = 0x6afedc
    cout << b-a << endl; // 2
}
```

Dynamic Array

- **Dynamic Array** variable of type T is a **pointer** to the **starting address** of consecutive block of type T
- Let A be a dynamic array of int
 - A[x] refer to the x^{th} block starting from A
- Static array works in the same way, that's why accessing A[x] is very fast for an array
 - It just refers to the address A[x] is $*(A + x * \text{sizeof}(T))$

new and delete operator

- For a datatype T, new T *allocates* a block with a size of T and then call a *constructor* of T, return the address of that memory
- For a datatype T, new T[n] *allocates* n blocks of T and then call a *constructor* of T in each block, return the address of the first block
 - For example, we use new int[10] to create a dynamic int array of 10 elements
- For a pointer X, delete X calls the *destructor* and then *de-allocates* memory pointed by X
- For a dynamic array X, delete [] calls the *destructor* of all blocks in the dynamic array and *de-allocates* the memory allocated by X

Example

```
class test {
public:
    // constructor
    test() : data() {cout << "created" << endl;}
    // destructor
    ~test() {cout << data << " destroyed " << endl;}
    int data;
};

int main() {
    test *a, *b;    // pointers of test

    a = new test;    // created new test pointed by a
    a->data = 10;    // (*a).data = 10
    cout << a->data << endl;    // 10
    delete a;        // destroyed

    b = new test[4]; // created 4x
    b[0].data = 10;
    b[1].data = 20;
    b[2].data = 30;
    b[3].data = 40;
    delete [] b;     // deleted 4x
}
```

Memory Leak

- For everything, this is created by new, we must call delete on it
- If you do not, that memory is not deleted until all memory is used up

```
#include <iostream>
using namespace std;

void leaked() {
    int *a;
    a = new int[2000];
}

int main() {
    for (int i=0; i<1000000; i++) {
        int *a;
```

```

    cout << i << endl;
    leaked();
}
}
// terminate called after throwing an instance of 'std::bad_alloc'

```

Smart Pointer (NOT A SUBJECT OF THIS COURSE)

- A better way is to use C++ smart pointer `shared_ptr(T)`
- Smart pointer is a pointer that can delete itself when it go out of scope
- Similar concept to [Java Garbage Collection](#)
- Still possible to have memory leak

vector.h

Version 0.1

- Start with vector that can do `push_back`, `pop_back`, and `[]`
- Also with custom constructor

```

namespace CP {
template <typename T>
class vector {
protected:
    T *mData;
    size_t mCap;
    size_t mSize;

    void rangeCheck(int n) {...}
    void expand(size_t capacity) {...}
    void ensureCapacity(size_t capacity) {...}
public:
    vector() {...}
    vector(size_t capacity) {...}
    ~vector() {...}
    // access
    T& at(int index) {...}
    T& operator[](int index) {...}
    // modifier
    void push_back(const T& element) {...}
    void pop_back() {...}
};
}

```

Basic Constructor

```

template <typename T>
class vector {
protected:
    T *mData;
    size_t mCap;
    size_t mSize;

public:
    vector() {
        int cap = 1;
        mData = new T[cap]();
        mCap = cap;
        mSize = 0;
    }
}

```

```
vector(size_t cap) {
    mData = new T[cap]();
    mCap = cap;
    mSize = cap;
}
```

```
vector<int> v;
```

```
vector<int> w(5);
```

```
mSize mCap mData
0      1    [ ]
```

```
mSize mCap mData
5      5    [0,0,0,0,0]
```

Destructor

- Since we new mData, we have to delete it
 - Or face a memory leak problem

```
~vector() {
    delete [] mData;
}
};
```

Object Life Cycle

- Normal object
 - Object is created (constructor called) when declared
 - Object is destroyed (destructor called) when go out of scope
- Object created by new (both new T or new T[]) Object is created when new Object is destroyed when delete

```
class test {
public:
    // constructor
    test() : data() {cout << "created" << endl;}
    // destructor
    ~test() {cout << data << " destroyed " << endl;}
    int data;
};
```

```
int main() {
    cout << "-- Life cycle --" << endl;
    cout << "- normal cycle -" << endl;
    test u;
    u.data = 99;
    for (int i=0; i<5; i++) {
        test t;
        t.data = i*10;
    }
}
```

Output

```
-- Life cycle --
- normal cycle -
created      <-- u
created
0 destroyed
created
10 destroyed
created
20 destroyed
```

```

created
30 destroyed
created
40 destroyed
99 destroyed  <-- u

```

Accessing Data

- The return type is T& which is a reference
- Same deal as pass-by-reference, this is called **return-by-reference**
 - So we can do `v[i] = 30` or `v[i]++`
- What is returned is actually that variable
- Also notice the difference between `at()` and operator `[]`

```

template <typename T>
class vector {
protected:
    T *mData;
    size_t mCap;
    size_t mSize;

    void rangeCheck(int n) {
        if (n < 0 || (size_t)n >= mSize) {
            throw std::out_of_range("index out of range");
        }
    }

public:
    T& at(int index) {
        rangeCheck(index);
        return mData[index];
    }

    T& operator[](int index) {
        return mData[index]; // return the variable
    }
};

```

Add, remove data

- `push_back` first check if we have reserved space
 - If not, we expand
- Then, the data is put to `mData[mSize]`
- Removing Data is done by just reduce the size

```

template <typename T>
class vector {
protected:
    T *mData;
    size_t mCap;
    size_t mSize;
    void expand(size_t capacity) {
        T *arr = new T[capacity](); // create a new dynamic array
        for (size_t i = 0; i < mSize; i++) {
            arr[i] = mData[i]; // move all data
        }
        delete [] mData;
        mData = arr; // delete old data and point to new one
        mCap = capacity;
    }
};

```

```

    }
    void ensureCapacity(size_t capacity) {
        if (capacity > mCap) {
            size_t s = (capacity > 2 * mCap) ? capacity : 2 * mCap; // double the size
            expand(s);
        }
    }
public:
    void push_back(const T& element) {
        ensureCapacity(mSize+1);
        mData[mSize++] = element; // add size and put element
    }
    void pop_back() {
        mSize--;
    }
};

```

Vector Constructors

Problem of v0.1

- Copy constructor and assignment operator is incorrect
 - It is auto generate to copy all variables (but not the data it points to)
- Rule of three in c++
 - Consider destructor, copy constructor, assignment operator
 - If any of them is written in the code, we mostly need all of them
- Since c++11, it's rule of four an a half

```

int main() {
    CP::vector<int> w(5);

    for (int i=0; i<5; i++) w[i] = i*10;
    CP::vector<int> x(w); // this creates shallow copy of w
    CP::vector<int> y = w;
    x[3] = -1; // changes both w and y
    cout << y[3] << endl; // -1
    cout << w[3] << endl; // -1
}

```

v0.2, add small access functions

- empty and size also exists in other data structure
- size_t is non-negative integer type

```

bool empty() const {
    return mSize == 0;
}

size_t size() const {
    return mSize;
}

size_t capacity() const {
    return mCap;
}

```

v0.2 adding copy constructor & assignment operator

```

// copy constructor
vector(const vector<T>& a) {
    mData = new T[a.capacity()]();
}

```



```

    mCap = a.capacity(); // copies only value
    mSize = a.size();
    for (size_t i=0; i<a.size(); i++) {
        mData[i] = a[i];
    }
}

// copy assignment operator
vector<T>& operator=(vector<T> &other) {
    // protect against self-destruct
    if (mData != other.mData) {
        // delete current data
        delete [] mData;
        // copy the new data
        mData = new T[other.capacity()]();
        mCap = other.capacity();
        mSize = other.size();
        for (size_t i=0; i<a.size(); i++) {
            mData[i] = a[i];
        }
    }
}

```

Copy-and-swap

- Utilize written copy-constructor and destructor
- Shorter code

```

// copy assignment operator using copy-and-swap idiom
vector<T>& operator=(vector<T> other) { // notice the pass-by-value!!!
    // other is copy-constructed which will be destruct at the end of this scope
    // we swap the content of this class to the other class and let it be destructed
    using std::swap;
    swap(this->mSize, other.mSize);
    swap(this->mCap, other.mCap);
    swap(this->mData, other.mData);
    return *this;
}

```

Iterator, Insert, Erase, & Analysis

v0.3 Iterator and typedef keyword

```

template <typename T>
class vector {

protected:
    T *mData;
    size_t mCap;
    size_t mSize;
protected:
    typedef T* iterator; // let iterator be type name of T pointer

    // iterator
    iterator begin() {
        return &mData[0];
    }

    iterator end() {

```

```

        return begin()+mSize;
    }
}

```

- See that pointer works just like how `std::vector::iterator` works
- In fact, iterator is actually a pointer
- typedef keyword allows us to map a type name
 - `CP::vector<int>::iterator` is `int*`
 - `CP::vector<bool>::iterator` is `bool*`

insert

- `push_back` actually call `insert(end(), element)`
- Question: why we need `pos`?

```

iterator insert(iterator it, const T& element) {
    size_t pos = it - begin();
    ensureCapacity(mSize+1);
    for (size_t i = mSize; i > pos; i--) {
        mData[i] = mData[i-1];
    }
    mData[pos] = element;
    mSize++;
    return begin()+pos;
}

```

```

void push_back(const T& element) {
    insert(end(), element);
}

```

erase

- See that both `insert` and `erase` also change `mSize`

```

void erase(iterator it) {
    while((it+1)!=end()) {
        *it = *(it+1);
        it++;
    }
    mSize--;
}

```

Exercise

- Read the following function and see how it works in `vector.h`
 - `resize`, `clear`
 - non-stl function
 - `insert_by_pos`
 - `erase_by_pos`
 - `erase_by_value`
 - `contains`
 - `index_of`
- Read in [here](#)

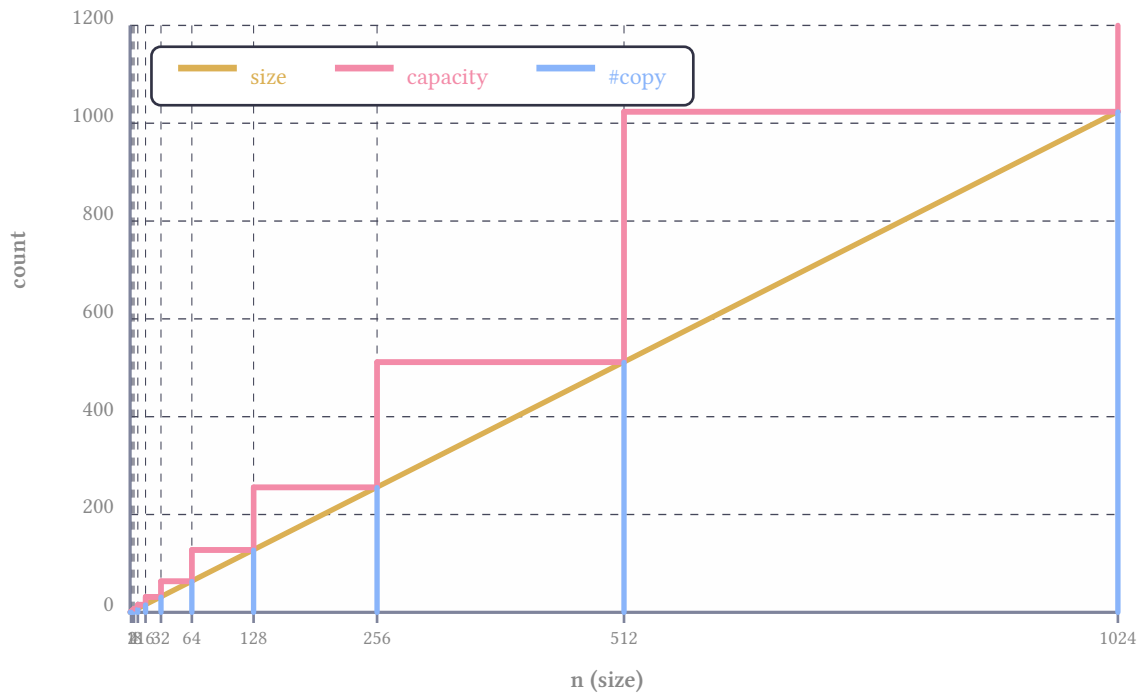
Will do this part later

Analysis of how many data is copied by `push_back`

- When full, `push_back` have to move all data to a new dynamic array
- `ensureCapacity` double the size

Size and Capa & Copy Count

- How much copy we need?



Lecture 10

Stack

Intro

- Now we will create less complex data structure `CP::stack`
- Just like a vector without iterator, insert, erase, resize, at end operator[]
 - Add `top()` which is just a shorthand of looking at the last element
- That's it, really

Key Idea

- The data is stored in the same way as a vector
 - The first element of `mData` is the bottom of stack while the last element is the top of the stack
- We just take `vector.h` and remove unnecessary function

```

| |   stack
| |
| A |   mSize mCap   mData
| X |   4     5     [A, B, X, A]
| B |
| A |
|___|

```

stack.h

```

namespace CP {
    template <typename T>
    class stack {
    protected:
        T *mData;
        size_t mCap;
        size_t mSize;
        void expand(size_t capacity) {...}
        void ensureCapacity(size_t capacity) {...}
    };
}

```

```

    public:
        // constructor
        stack(const stack<T>& a) {...}
        stack() {...}
        stack<T>& operator= {...}
        ~stack() {...}
        // capacity operation
        bool empty() const {...}
        size_t size() const {...}
        // access
        const T& top() const {
            return mData[mSize-1];
        }
        // modifier
        void push(const T& operator) {...} // This is push_back
        void pop() {...} // This is pop_back
};
}

```

Speed of each operation

- All read operation always take constant time
 - size(), top() simply return something that is directly accessible
- All modify operation also take constant time
 - push() is constant on average (same as push_back of vector)
 - pop() is always constant

Stack by Vector

- Instead of writing our own function, there is another way to write a stack
- We simply use **vector** as our sole data member
- Benefit: **code reuse**
- Drawback: **almost none** except that we need one more layer of function call

```

namespace CP {
    template <typename T>
    class stack {
    protected:
        vector<T> v;
    public:
        // default constructor
        stack() : v() { }
        // capacity function
        bool empty() const { return v.empty(); }
        size_t size() const { return v.size(); }
        // access
        const T& top() const { return v.back(); }
        // modifier
        void push(const T& element) { v.push_back(element); }
        void pop() { v.pop_back(); }
    };
}

```

Lecture 11

CP::queue

Will the circle be unbroken?

Intro

- Queue, unlike stack, require more sophisticated technique to achieve fast performance
- We start by writing a simple class that just work (slowly)
- Then we try to improve it

Key Idea

- Just like stack, we will use the same format as vector, using dynamic array to store data
- However we have to somehow manage how we works with `front()` and `back()` of the queue

v0.1 simple implementation of the queue

- To illustrate this idea, we will use a **vector** as our data member
- `push(e)` is simply `v.push_back(e)`, this is fast
- `front()` is `v[0]`, `back()` is `v[v.size()-1]`, this is also fast
- `pop()` is `v.erase(v.begin())`, this is slow (always proportional to `v.size()`)
 - Unlike `std::queue` which has very fast `pop()`

```
namespace CP {
    template <typename T>
    class queue {
    protected:
        std::vector<T> v;
    public:
        // default constructor
        queue() : v() { }
        // capacity function
        bool empty() const { return v.empty(); }
        size_t size() const { return v.size(); }
        // access
        const T& front() const { return v[0]; }
        const T& back() const { return v[v.size()-1]; }
        // modifier
        void push(const T& element) { v.push_back(element); }
        void pop() { v.erase(v.begin()); }
    };
}
```

v0.1 example

```
CP::queue<int> q;    q  v
                     mSize mCap mData
                       0    1    ----> [ ]

q.push(10);          q  v
                     mSize mCap mData
                       1    1    ----> [10]

q.push(20);          q  v
                     mSize mCap mData
                       2    2    ----> [10, 20]

q.push(30);          q  v
                     mSize mCap mData
                       3    4    ----> [10, 20, 30, ]

q.pop();             q  v
                     mSize mCap mData
                       2    4    ----> [20, 30, , ]
```

v0.2 faster queue

- Add more data member `mFront`, initialized as 0
- `push(e)` is simply `v.push_back(e)`, this is fast
- `front()` is `v[mFront]`, `back()` is `v[v.size()-1]`, this is also fast

- `pop()` is `mFront++`, this is fast
 - However, we don't really remove anything when `pop`

```
#include <vector>

namespace CP {
    template <typename T>
    class queue {
    protected:
        std::vector<T> v;
        int mFront;
    public:
        // constructor
        queue() : v(), mFront() {}
        // capacity function
        bool empty() const { return v.empty(); }
        size_t size() const { return v.size() - mFront; }
        // access
        const T& front() const { return v[mFront]; }
        const T& back() { return v[v.size()-1]; }
        // modifier
        void push(const T& element) { v.push_back(element); }
        void pop() { mFront++; }
    };
}
```

v0.2 example

CP::queue<int> q;	q	v	
	mFront	mSize	mCap mData
	0	0	1 ----> []
q.push(10);	q	v	
	mFront	mSize	mCap mData
	0	1	1 ----> [10]
q.push(20);	q	v	
	mFront	mSize	mCap mData
	0	2	2 ----> [10, 20]
q.push(30);	q	v	
	mFront	mSize	mCap mData
	0	3	4 ----> [10, 20, 30,]
q.pop();	q	v	
	mFront	mSize	mCap mData
	1	3	4 ----> [10, 20, 30,]

Problem with v0.2

- Fast but **use too many space**
- Queue grows according to **how many time push is called**
 - regardless of how many pop is called
- The data stored in the vector can be **much larger** than the actual data in the queue
- Does not really work in real world

```
for (int i=0; i<1000000; i++) {
    q.push(i);
    q.pop();
}
std::cout << q.size() << std::endl;
```

Circular Queue

Final Idea

```

mFront  mSize  mCap  mData          *
  2      2     4      ----> [ , , 30, 40]
q.push(50);
mFront  mSize  mCap  mData          *
  2      3     4      ----> [50, , 30, 40]
q.push(60);
mFront  mSize  mCap  mData          *
  2      4     4      ----> [50, 60, 30, 40]
q.pop();
mFront  mSize  mCap  mData          *
  3      3     4      ----> [50, 60, , 40]
q.push(99);
mFront  mSize  mCap  mData          *
  3      4     4      ----> [50, 60, 99, 40]
q.push(1);
mFront  mSize  mCap  mData          * \ \ \
  0      5     4      ----> [40, 50, 60, 99, 1, , , ]

```

- We take v0.2 and **reuse** the area at the beginning of mData
 - Expand when necessary
 - Re-arrange when expand

Circular Queue

- We can think of mData to be circular
 - End of the last element of the mData is connected to the first element
- Consider i^{th} element
 - the next element is $(i+1) \% \text{mCap}$
 - The previous element is $(i-1+\text{mCap}) \% \text{mCap}$
 - Next k element is $(i+k) \% \text{mCap}$

Circular Queue Implementation

queue.h

```

namespace CP {
    template <typename T>
    class queue {
    protected:
        T *mData;
        size_t mCap;
        size_t mSize;
        size_t mFront;
        void expand(size_t capacity) {...}
        void ensureCapacity(size_t capacity) {...}
    public:
        // constructor - almost the same but have to take care of mFront
        queue(const queue<T>& a) {...}
        queue() {...}
        queue<T>& operator=(queue<T> other) {...}
        ~queue() {...}
        // capacity function - same as vector
        bool empty() const {...}
        size_t size() const {...}
        // access - circular queue implementation
        const T& front() const {...}
        const T& back() const {...}

```

```

        // modifier
        void push(const T& element) {...}
        void pop() {...}
    };
}

```

Ctor, Dtor, copy

```

template <typename T>
class queue {
protected:
    T *mData;  size_t mCap;  size_t mSize;  size_t mFront;
public:
    // default constructor
    queue() : mData(new T[1]()), mCap(1),
             mSize(0), mFront(0) { }
    // copy constructor
    queue(const queue<T>& a) : mData(new T[a.mCap]()), mCap(a.mCap),
                            mSize(a.mSize), mFront(a.mFront) {
        for (size_t i = 0; i < a.mCap; i++) {
            mData[i] = a.mData[i];
        }
    }
    // copy assignment operator
    queue<T>& operator=(queue<T> other) {
        using std::swap;
        swap(mSize, other.mSize);
        swap(mCap, other.mCap);
        swap(mData, other.mData);
        swap(mFront, other.mFront);
        return *this;
    }
    ~queue() {
        delete [] mData;
    }
};

```

front(), back(), pop()

Continue at vid 52 5.48